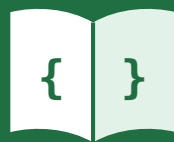

Module 00 — Setup and Modern Java (21 → 27)

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Required Tools	2
2.2 Verifying the Setup	2
2.3 How to Use the Repository	3
2.4 What Changed from Java 21 to 27?	3
3. Step-by-Step Examples	4
3.1 Records	4
3.2 Sealed Types and the Exhaustive Switch	5
3.3 Record Patterns	6
3.4 Text Blocks and var	7
3.5 Virtual Threads	8
3.6 Scoped Values	9
3.7 Sequenced Collections	9
3.8 Stream Gatherers	10
3.9 Compact Source Files	11
3.10 Structured Concurrency (Preview)	12
4. Common Mistakes and Best Practices	13
5. Summary	13
6. Further Reading	13

1. Learning Goals

By the end of this module you will be able to:

- Install JDK 27, Docker and your IDE, and run the course repository
- Use the module layout (lesson / exercise / solution) and the exercise workflow
- Write data-oriented, safe code with records, sealed types and pattern matching
- Write more readable code with text blocks, `var`, sequenced collections and stream gatherers
- Scale blocking code with virtual threads and carry context with scoped values
- Run single-file Java programs as compact source files
- Know what preview features (structured concurrency) are and how to enable them

Prerequisites: basic Java (classes, interfaces, collections, lambdas) · **Estimated time:** 3 hours

2. Concepts

2.1 Required Tools

Tool	Why?	Note
JDK 27	Every module compiles with Java 27	Oracle JDK, Amazon Corretto or any OpenJDK 27 distribution
Docker	Databases and other services	Give Docker Desktop at least 8 GB RAM
IDE	Writing code, running tests	A current IntelliJ IDEA or VS Code (with the Java extensions) that supports Java 27
Git	Getting the repository	Any version

You do not need to install Maven. The repository ships the Maven Wrapper (`./mvnw`), which downloads the right Maven version by itself.

2.2 Verifying the Setup

```

java -version                                # should print "27"
export JAVA_HOME=$(/usr/libexec/java_home -v 27) # macOS; Linux/Windows: point
↪ JAVA_HOME to JDK 27
./mvnw -v                                    # should print "Java version: 27"
docker info --format '{{.ServerVersion}}'      # is Docker running?

```

! Warning

Even if `java -version` prints 27, Maven may still run on another JDK. Check the **Java version** line of `./mvnw -v`. If it is wrong, the build stops with “This course needs JDK 27”.

2.3 How to Use the Repository

Every module under `modules/NN-topic/` has the same layout:

Folder	Content
<code>lesson/</code>	The lesson’s examples and tests. Always runs, always green.
<code>exercise/</code>	Starter code for the exercises. TODOs wait for you, tests start red.
<code>solution/</code>	The reference solution. Its tests are identical to <code>exercise/</code> .
<code>docs/</code>	Lesson notes and exercises, in Turkish and English, as MD and PDF

Exercise workflow: read `docs/en/exercises.md`, complete the TODOs in `exercise/`, and turn the tests green with `./mvnw -Pexercises -pl modules/<module>/exercise test`.

2.4 What Changed from Java 21 to 27?

Java ships a new release every six months. A feature first arrives as a **preview**, matures through feedback and then becomes **final**. Preview features are disabled by default and need the `--enable-preview` flag.

Feature	Status (Java 27)	Section
Records, sealed types	Final (16, 17)	3.1, 3.2

Feature	Status (Java 27)	Section
Pattern matching for switch, record patterns	Final (21)	3.2, 3.3
Unnamed variables <code>_</code>	Final (22)	3.2, 3.3
Text blocks, <code>var</code>	Final (15, 10)	3.4
Virtual threads	Final (21); no synchronized pinning since 24	3.5
Scoped values	Final (25)	3.6
Sequenced collections	Final (21)	3.7
Stream gatherers	Final (24)	3.8
Compact source files, instance <code>main</code>	Final (25)	3.9
Structured concurrency	Preview (7th preview, JEP 533)	3.10

Java 27 also makes G1 the default garbage collector in all environments (JEP 523) and enables compact object headers by default (JEP 534). Both save memory and time without any code change.

i Note

The course's lesson code uses **final** features only. Preview features appear only in this module, in a separate Maven profile (section 3.10).

3. Step-by-Step Examples

To run every example in lesson order:

```
./mvnw -pl modules/00-setup-modern-java/lesson spring-boot:run
```

The examples live below the `com.springbootedu.setupmodernjava` package and use the **Bookstore** domain that we will use throughout the course.

3.1 Records

Goal: define immutable data carriers in a single line.

For a record, the fields, constructor, accessors (`amount()`), `equals`, `hashCode` and `toString` are generated. A **compact constructor** validates and normalises without repeating the parameter list:

```
public record Money(BigDecimal amount) {

    public static final Money ZERO = new Money(BigDecimal.ZERO);

    public Money {                                     // compact constructor: validate
        ↪ & normalise
        Objects.requireNonNull(amount, "amount");
        if (amount.signum() < 0) {
            throw new IllegalArgumentException("Amount must not be negative: " + amount);
        }
        amount = amount.setScale(2, RoundingMode.HALF_UP); // assigns the field after
        ↪ this block
    }

    public static Money of(String amount) {
        return new Money(new BigDecimal(amount));
    }

    public Money plus(Money other) {
        return new Money(amount.add(other.amount)); // records are immutable →
        ↪ return a new value
    }
}
```

Expected output:

```
== 3.1 Records
Money.of("12.5") = 12.50, ISBN = 9780134685991
```

Its test: `records/MoneyTest` covers value equality, validation and immutability.

3.2 Sealed Types and the Exhaustive Switch

Goal: tell the compiler every subtype of a type, and never forget a case.

A sealed interface lists who may implement it with `permits`:

```
public sealed interface Discount permits PercentageDiscount, FixedDiscount, NoDiscount {
}
```

Because the compiler knows every subtype, the switch is **exhaustive**. No default branch is needed. If you add a new kind of discount, this method stops compiling until you handle it:

```
public static Money apply(Discount discount, Money price) {
    return switch (discount) {
        // exhaustive: every permitted type
        ↪ is covered
        case PercentageDiscount p -> price.percentOff(p.percent());
        case FixedDiscount f -> price.minusOrZero(f.amount());
        case NoDiscount _ -> price; // "_" = unnamed variable (we don't
        ↪ need it)
    };
    // Add a fourth Discount type and this method stops compiling until you handle it.
}
```

Expected output:

```
== 3.2 Sealed types & exhaustive switch
%10 indirim / 10% off → 180.00
30.00 TL indirim / 30.00 TRY off → 170.00
İndirim yok / No discount → 200.00
```

Its test: sealed/DiscountTest

3.3 Record Patterns

Goal: take nested records apart in one step and add conditions with when.

A record pattern binds the components of an object directly to variables. `_` skips the components we do not need. Branches are tried **top to bottom**, so special cases come first:

```
public static Money total(OrderLine line) {
    return switch (line) {
        case OrderLine(_, int quantity) when quantity == 0 -> Money.ZERO;
        case OrderLine(Book(_, Money price, Format format), int quantity) when format ==
        ↪ Format.EBOOK ->
            price.times(Math.min(quantity, 3)); // e-books: pay for
        ↪ at most 3 copies
        case OrderLine(Book(_, Money price, _), int quantity) when quantity >= 10 ->
            price.times(quantity).percentOff(20); // bulk order: 20% off
        case OrderLine(Book(_, Money price, _), int quantity) -> price.times(quantity);
    };
}
```

Expected output:

```
== 3.3 Record patterns
10 × paperback = 800.00
10 × e-book    = 120.00
```

Its test: patterns/OrderLinePricingTest

3.4 Text Blocks and var

Goal: write multi-line text (JSON, SQL) readably, and stop repeating type names in local variables.

A text block starts with `"""`. The common indentation to the left of the closing quotes is removed automatically:

```
public static String toJson(String title, String author, int year) {
    return """
        {
            "title": "%s",
            "author": "%s",
            "year": %d
        }"""
    ↪ .formatted(title, author, year); // indentation left of the closing
    ↪ quotes is removed
}
```

`var` is for **local** variables whose type is obvious from the right-hand side:

```
public static String receipt(List<String> titles) {
    var lines = new StringBuilder(); // var: the type is obvious from the
    ↪ right-hand side
    for (var i = 0; i < titles.size(); i++) {
        lines.append(i + 1).append(". ").append(titles.get(i)).append('\n');
    }
    return """
        KİTAPÇI / BOOKSTORE
        %sToplam / Total: %d kitap / books
        """
    ↪ .formatted(lines, titles.size());
}
```

Expected output:

```
== 3.4 Text blocks & var
{
    "title": "Effective Java",
    "author": "Joshua Bloch",
    "year": 2018
}
```

Its test: textblocks/CatalogExportTest

3.5 Virtual Threads

Goal: scale blocking code (waiting for the network or a database) to thousands of concurrent tasks without switching to reactive programming.

Virtual threads are lightweight threads managed by the JVM. While a virtual thread waits (e.g. `Thread.sleep`, a socket read), the operating-system thread underneath is released and runs other work. That makes a new virtual thread per task cheap:

```
public static Result lookupAll(int count, Duration latency) {
    long start = System.nanoTime();
    List<Future<Integer>> prices = new ArrayList<>();

    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { // one new
        ↪ virtual thread per task
        for (int i = 0; i < count; i++) {
            prices.add(executor.submit(() -> fetchPrice(latency))); // plain
        }
        ↪ blocking code
    }
    ↪ for all tasks
    // close() waits

    int completed = (int) prices.stream().filter(Future::isDone).count();
    return new Result(completed, Duration.ofNanos(System.nanoTime() - start));
}

private static int fetchPrice(Duration latency) throws InterruptedException {
    Thread.sleep(latency); // simulates a slow remote call; the carrier thread
    ↪ is released meanwhile
    return 42;
}
```

Expected output (the duration depends on the machine):

```
== 3.5 Virtual threads
10000 blocking calls of 100 ms took 135 ms
```

Sequentially this would take $10,000 \times 100 \text{ ms} \approx 17 \text{ minutes}$.

Its test: `virtualthreads/PriceLookupTest`

★ Tip

In Spring Boot, `spring.threads.virtual.enabled=true` runs web requests, `@Async` tasks and scheduled jobs on virtual threads. Every module of this course enables it (`application.yaml`).

3.6 Scoped Values

Goal: share a context such as “the current customer” without passing it as a parameter through every layer.

ScopedValue is an immutable, bounded alternative to ThreadLocal. The value is only bound while the `run(...)` block executes and disappears by itself afterwards. There is nothing to forget to clean up:

```
public final class CurrentCustomer {

    private static final ScopedValue<String> EMAIL = ScopedValue.newInstance();

    private CurrentCustomer() {}

    public static void runAs(String email, Runnable action) {
        ScopedValue.where(EMAIL, email).run(action);    // bound only while action runs
    }

    public static String email() {
        return EMAIL.orElse("anonymous");              // unbound outside the scope
    }
}
```

Code deep in the call chain reads the value without taking a parameter:

```
public void checkout(String isbn) {
    log.add(CurrentCustomer.email() + " bought " + isbn);    // no "customer" parameter
    ↪ needed
}
```

Expected output:

```
== 3.6 Scoped values
ayse@example.com bought 978-0-13-468599-1
anonymous bought 978-1-61729-757-1
```

Its test: `scopedvalues/AuditLogTest`

3.7 Sequenced Collections

Goal: access the first/last element and the reverse order of ordered collections through one uniform API.

The `SequencedCollection`, `SequencedSet` and `SequencedMap` interfaces bring `getFirst()`, `getLast()`, `addFirst()`, `removeLast()` and `reversed()`. They fit a “recently viewed books” list perfectly:

```
public class RecentlyViewed {

    private final SequencedSet<String> isbnns = new LinkedHashSet<>();
    private final int limit;

    public RecentlyViewed(int limit) {
        this.limit = limit;
    }

    public void view(String isbn) {
        isbnns.addFirst(isbn);           // already present? LinkedHashSet moves it to
        ↪ the front
        while (isbnns.size() > limit) {
            isbnns.removeLast();         // drop the oldest
        }
    }

    public List<String> newestFirst() {
        return List.copyOf(isbnns);
    }

    public List<String> oldestFirst() {
        return List.copyOf(isbnns.reversed()); // a view, no copy of the set
    }

    public Optional<String> latest() {
        return isbnns.isEmpty() ? Optional.empty() : Optional.of(isbnns.getFirst());
    }
}
```

Expected output:

```
== 3.7 Sequenced collections
newest first [D, A, C], oldest first [C, A, D]
```

Its test: `collections/RecentlyViewedTest`

3.8 Stream Gatherers

Goal: perform intermediate operations that `map/filter` cannot express (batching, sliding windows, running totals) inside a stream.

`Stream.gather(...)` lets you plug in your own intermediate operation. The JDK also ships ready-made gatherers:

```
public static List<List<String>> batches(List<String> isbns, int size) {
    return isbns.stream().gather(Gatherers.windowFixed(size)).toList();    // [a,b]
    ↪ [c,d] [e]
}

public static List<Double> movingAverage(List<Integer> dailySales, int days) {
    return dailySales.stream()
        .gather(Gatherers.windowSliding(days))                            //
        ↪ [10,20,30] [20,30,40]
        .map(window ->
            ↪ window.stream().mapToInt(Integer::intValue).average().orElse(0))
        .toList();
}

public static List<Integer> runningTotal(List<Integer> sales) {
    return sales.stream().gather(Gatherers.scan(() -> 0, Integer::sum)).toList();    // 5,
    ↪ 15, 35
}
```

Expected output:

```
== 3.8 Stream gatherers
batches [[a, b], [c, d], [e]]
moving average [20.0, 30.0]
running total [5, 15, 35]
```

Its test: `gatherers/SalesStatisticsTest`

3.9 Compact Source Files

Goal: write and run a small program without a class, `public static` or a build tool.

A file may contain just a main method. Every package of the `java.base` module is imported automatically, and `IO.println` writes to the console:

```
void main(String[] args) {
    String name = args.length > 0 ? args[0] : "Java 27";
    var books = List.of("Effective Java", "Java Puzzlers", "Modern Java in Action");    //
    ↪ java.base is imported

    IO.println("Merhaba %s! / Hello %s!".formatted(name, name));
    IO.println(books.size() + " kitap / books: " + books);
}
```

Run it:

```
cd modules/00-setup-modern-java/lesson
java src/scripts/HelloBookstore.java Ayşe
```

Expected output:

```
Merhaba Ayşe! / Hello Ayşe!
3 kitap / books: [Effective Java, Java Puzzlers, Modern Java in Action]
```

Its test: compactsource/HelloBookstoreScriptTest really runs the file with the java command.

3.10 Structured Concurrency (Preview)

Goal: manage related concurrent tasks as one unit. If one fails the others are cancelled, and no thread is left “orphaned”.

A book’s price and reviews come from two services, fetched **at the same time**. The total time is the longest call, not the sum of both:

```
public BookDetails load(String isbn) throws InterruptedException, ExecutionException {
    try (var scope = StructuredTaskScope.open()) {                // default: all must
        ↪ succeed
        var price = scope.fork(() -> fetchPrice(isbn));           // runs in its own
        ↪ virtual thread
        var reviews = scope.fork(() -> fetchReviews(isbn));

        scope.join();      // waits for both; if one fails, the other is cancelled and
        ↪ ExecutionException is thrown

        return new BookDetails(price.get(), reviews.get());
    }
}
```

This code lives in a separate source folder (src/preview/java) and compiles only with the preview profile:

```
./mvnw -Ppreview -pl modules/00-setup-modern-java/lesson verify
```

Its test: src/preview-test/.../BookDetailsLoaderTest checks that both calls run in parallel and that one failure stops the whole operation.

!! Caution

Preview APIs can change between releases. For example, the exception thrown by `join()` changed between previews. Do not use preview features in production code.

4. Common Mistakes and Best Practices

! Warning

Do not add a default branch to a switch over a sealed type. `default` stops the compiler from warning you when a new subtype is added.

- **Do:** make value objects (money, ISBN, address) records and put validation into the compact constructor.
- **Don't:** use records as JPA entities. Entities must be mutable (Module 06).
- **Do:** never pool virtual threads; create a new one per task (`newVirtualThreadPerTaskExecutor`).
- **Don't:** try to limit concurrency through a thread pool size. Use a Semaphore if you must.
- **Do:** prefer `ScopedValue` over `ThreadLocal` for request context in new code.
- **Don't:** use `var` where the type is not obvious (`var x = service.process();`).
- **Do:** use compact source files for one-off tools and experiments. Use a normal Maven project for applications.

5. Summary

- The course runs on JDK 27, Docker and the Maven Wrapper. `./mvnw -v` must show the right JDK.
- Every module holds lesson/ examples, exercise/ starters and solution/ answers.
- Records, sealed types and pattern matching together build safe, concise data models.
- Text blocks, `var`, sequenced collections and gatherers simplify everyday code.
- Virtual threads scale blocking code. Scoped values carry context safely.
- Preview features need `--enable-preview` and are not for production.

Next step: [Exercises](#)

6. Further Reading

- [JEP 395: Records](#) · [JEP 409: Sealed Classes](#)
- [JEP 441: Pattern Matching for switch](#) · [JEP 440: Record Patterns](#) · [JEP 456: Unnamed Variables & Patterns](#)
- [JEP 444: Virtual Threads](#) · [JEP 491: Synchronize Virtual Threads without Pinning](#)

- [JEP 506: Scoped Values](#) · [JEP 431: Sequenced Collections](#) · [JEP 485: Stream Gatherers](#)
- [JEP 512: Compact Source Files and Instance Main Methods](#)
- [JEP 533: Structured Concurrency \(Seventh Preview\)](#)
- [JDK 27 release page](#)